

CITYMIND

An Urban Intelligence System



FINAL REPORT

AI-2002 · Semester Project
Department of Computer Science

Members	Student ID
Haris Said	24I-0527
Anoosha Ahsan	24I-0831

Submitted To: Ms. Marium Hida
10 May, 2026

1. Overview

CityMind is a complete urban intelligence simulation built on a 20×20 grid-based graph. Five AI modules run in sequence on a single shared city graph, each consuming the output of the previous, to produce a city that plans its own layout, builds a minimum-cost road network, predicts crime risk, positions ambulances optimally, and reroutes an emergency medical team in real time around dynamic flood events.

This report documents what was implemented in the final submission, the specific design decisions made during development, the work split between both group members, and justifications for any deviations from the Phase 1 design document.

2. Shared City Graph

All five modules interact through a single shared CityGraph object. No module maintains its own copy of the city. The graph is the single source of truth for the entire system.

2.1 Node Structure

Each node represents one grid cell on the 20×20 grid and stores the following fields:

- **grid_position (row, col):** Serves as the unique identifier for each node.
- **location_type:** One of Residential, Hospital, School, Industrial, PowerPlant, AmbulanceDepot, or Empty.
- **population_density:** Assigned by Challenge 1 based on location type. Used as a feature in Challenge 5.
- **risk_index:** Defaults to 1.0. Updated by Challenge 5 to 1.5 (High), 1.2 (Medium), or 1.0 (Low).
- **accessibility:** Boolean flag. Defaults to True.

2.2 Edge Structure

Edges are not represented as a separate class. The CityGraph class maintains an adjacency dictionary at the graph level: `edges = {((row1,col1),(row2,col2)): cost}`. After Challenge 1 completes, every adjacent cell pair in the 4-directional grid is registered with a base cost of 0.8 if either endpoint is Residential, and 1.0 otherwise.

2.3 Effective Cost and Integration Mechanism

The central integration mechanism across all modules is the `get_edge_cost()` function:

Effective Cost Formula

$$\text{effective_cost}(u \rightarrow v) = \text{base_cost} \times \text{node_v.risk_index}$$

When Challenge 5 updates a node's `risk_index`, the effective cost of every edge leading to that node changes instantly at query time. Challenges 3 (GA fitness) and 4 (A* pathfinding) call `get_edge_cost()` for every edge expansion, so they automatically incorporate updated crime risk without any explicit crime logic in their own code. A blocked edge returns `float('inf')` from `get_edge_cost()` and is naturally avoided by both algorithms.

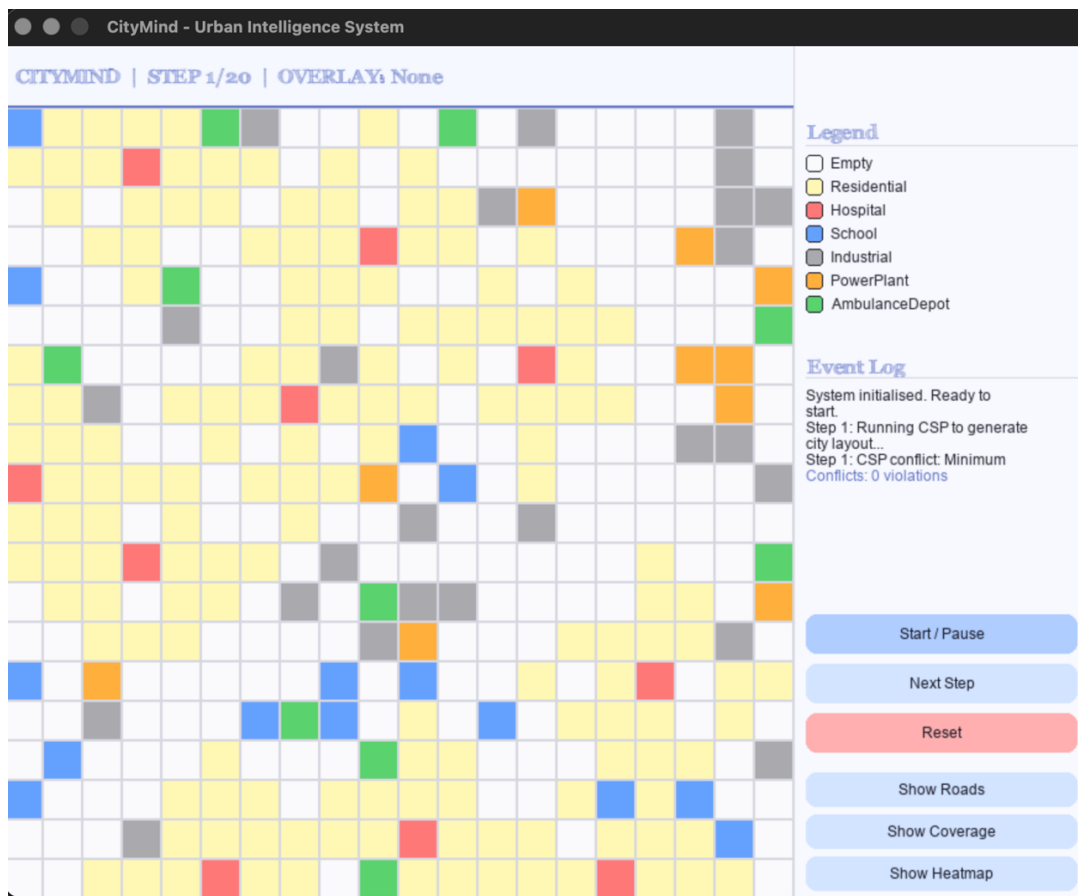
2.4 Observer-Style Updates

The graph implements a simple event model. Any module can register a callback for change events. When a road is blocked by the flood system, `check_path_affected()` is called on the medical team, which re-runs A* from the team's current position. This ensures that Challenge 4 reacts to changes without polling or any direct dependency between the flood system and the routing module.

3. Challenge 1 — City Layout Planning (CSP)

Challenge 1 populates the initially empty 20×20 grid by assigning location types to cells while satisfying three hard planning constraints:

- Industrial zones cannot be placed adjacent to Schools or Hospitals.
- Every Residential area must be within 3 road hops of at least one Hospital.
- Every Power Plant must be within 2 road hops of at least one Industrial zone.



3.1 Algorithm: Backtracking CSP with AC-3 and Minimum Conflicts Fallback

The solver runs in two phases. Phase 1 runs AC-3 arc consistency preprocessing to prune impossible values from cell domains before search begins, followed by backtracking with forward checking to place the 70 special-type nodes (10 Hospitals, 15 Schools, 25 Industrial zones, 10 Power Plants, 10 Ambulance Depots). Phase 2 fills the remaining cells with Residential nodes (placed only within 3 hops of a Hospital via BFS) and Empty.

A 5-second timeout is enforced on backtracking. If the timeout expires or no valid layout is found, the solver falls back to the Minimum Conflicts heuristic. Min-Conflicts initialises a random complete assignment with

exact type counts maintained throughout, then iteratively performs swap moves: a conflicted node is selected and swapped with the candidate that reduces total constraint violations the most. This preserves type counts exactly while minimising violations.

3.2 AC-3 Scope

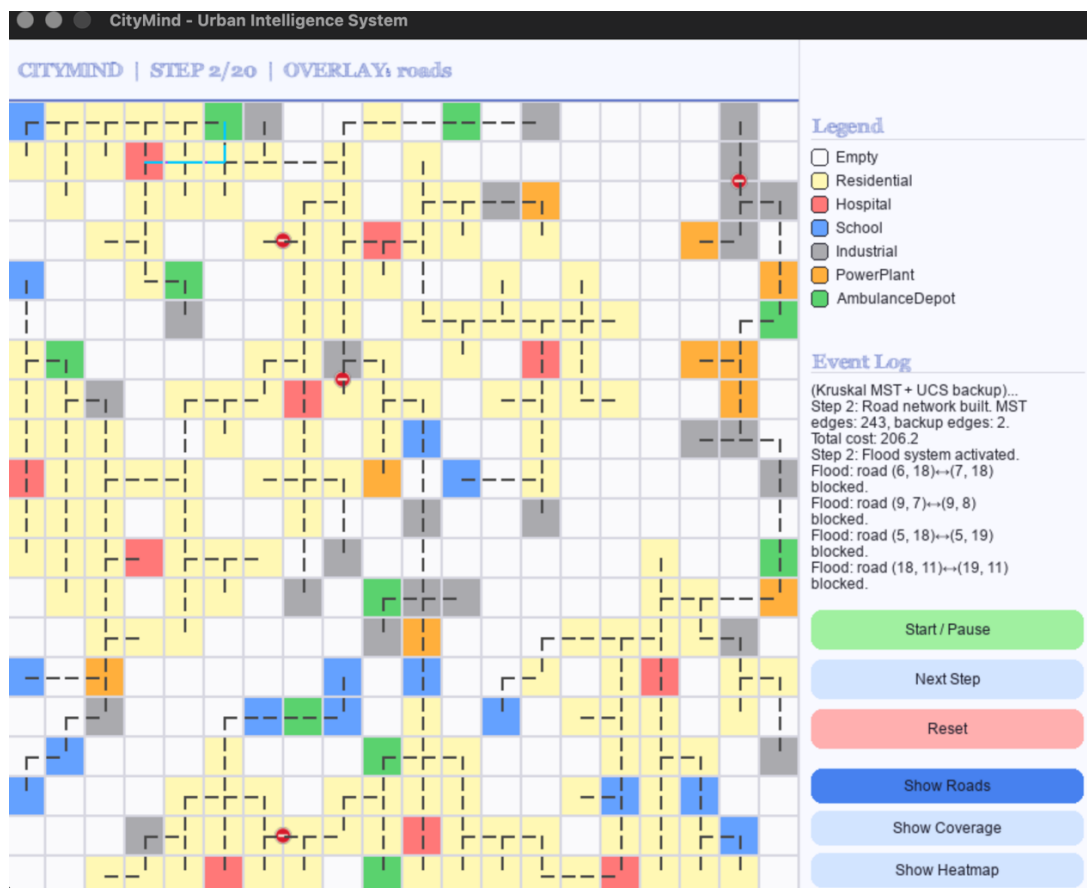
AC-3 is applied only to the Industrial adjacency constraint (Rule 1) because that is the only pairwise constraint between directly adjacent nodes. Rules 2 and 3 involve BFS over multiple hops and cannot be expressed as arc constraints efficiently, so they are checked during backtracking.

3.3 Conflict Detection and Reporting

`full_assignment_valid()` runs after Phase 2 completes and performs a full pass over all nodes, checking all three constraints. If any violation is found, the specific rule and the exact cell position are identified and returned to the event log. When Min-Conflicts runs and terminates with remaining violations, the count and nature of those violations is reported explicitly.

4. Challenge 2 — Road Network Optimisation (Kruskal's MST + UCS)

Challenge 2 builds the minimum-cost connected road network across all city nodes while guaranteeing two independent paths between the Primary Hospital and the Ambulance Depot.



4.1 Phase 1: Metric-Closure MST

Rather than running Kruskal's algorithm only on direct 4-directional grid edges, the implementation computes a metric-closure MST. UCS is run from every meaningful (non-Empty) node to compute shortest-path distances to every other meaningful node. These inter-node distances form the closure edges. Kruskal's algorithm then runs over these closure edges using a Union-Find (DSU) structure with path compression and union by rank, expanding each chosen closure edge back into the actual grid road corridor it represents.

This produces a significantly higher quality road network than naive grid-edge MST because it finds the cheapest way to connect locations that may be separated by empty space, rather than being forced to build roads through every intermediate empty cell.

4.2 Phase 2: UCS Redundancy Enforcement

An MST produces exactly one path between any two nodes, meaning a single road failure could cut off the Hospital from the Depot. After the MST is built, UCS is run over all non-MST grid edges (edges that physically exist on the grid but were not selected by Kruskal's) to find the cheapest additional path that creates a second independent, edge-disjoint route between the Primary Hospital and Ambulance Depot. The cheapest such path is added to the built road network.

The result — MST roads plus UCS redundancy roads — is stored in `graph.built_roads` and `graph.backup_roads` respectively, so the UI can render them in distinct colours.

5. Challenge 3 — Ambulance Placement (Genetic Algorithm)

Challenge 3 places 3 ambulances across Hospital and AmbulanceDepot nodes to minimise worst-case response distance — the maximum BFS distance from any Residential node to its nearest ambulance over the built road network.



5.1 GA Structure

Each chromosome is a tuple of 3 distinct node positions representing ambulance placements. The fitness function evaluates each chromosome by running BFS from each station over built_roads and recording the maximum nearest-ambulance distance across all Residential nodes. Lower fitness is better.

5.2 GA Operators

- **Population:** 60 chromosomes per generation.
- **Selection:** Tournament selection with a tournament size of 5. Five chromosomes compete per selection round; the one with the lowest fitness wins. Chosen over roulette wheel selection because it handles similar fitness values more robustly.
- **Crossover:** Single-point crossover at a random split point on the 3-position chromosome. Duplicate positions after crossover are repaired by replacing each duplicate with a randomly chosen valid station not already in the chromosome.
- **Mutation:** Random resetting mutation at 15% probability per gene, replacing one ambulance position with a different valid station. Appropriate for discrete grid positions where arithmetic mutation makes no sense.
- **Elitism:** The best chromosome from each generation is carried unchanged to the next.
- **Termination:** 300 generations or early stopping after 50 consecutive generations with no improvement.

5.3 Integration with Challenge 5

The GA fitness function calls `get_edge_cost()` for all BFS traversals, which incorporates the `risk_index` multipliers updated by Challenge 5. When Challenge 5 updates risk weights, ambulance placement

automatically responds because the fitness evaluation uses the live shared graph. This is why Challenge 5 runs before Challenge 3 in the simulation sequence.

6. Challenge 4 — Emergency Routing Under Changing Conditions (A*)

Challenge 4 routes a medical team from a Hospital or AmbulanceDepot starting position to rescue 5–8 trapped civilians placed on Residential nodes. Roads may flood at random intervals. The team must reach all civilians and the system must guarantee the shortest currently available path at all times.

6.1 A* Search Implementation

A* uses a min-heap open set of $(f, g, \text{position})$ tuples. The heuristic is Manhattan distance: $h(n) = |\text{row}_n - \text{row_goal}| + |\text{col}_n - \text{col_goal}|$. This heuristic is admissible on a 4-directional grid because the actual path cost can never be less than the straight Manhattan distance, guaranteeing that A* always finds the true shortest path.

A* routes over ALL grid edges (not just built roads). This is a deliberate design decision justified in the design document: emergency vehicles operate under conditions where reaching a civilian as fast as possible takes priority over staying on planned roads, and routing off-road prevents the team from becoming completely stuck when built roads flood.



6.2 Multi-Stop Civilian Ordering

Civilians are sorted nearest-first from the team's starting position using actual A* cost (not Manhattan distance) before the mission begins. After each rescue, remaining civilians are re-sorted by A* cost from the

team's current position, so the ordering always reflects the current road state including any floods that have occurred.

6.3 Dynamic Re-routing on Floods

The flood system is activated immediately after Challenge 2 completes, so roads can block at any point after roads are built — not only during Challenge 4. A random built road is selected every 6–10 seconds and blocked in the shared graph. When a flood fires, `check_path_affected()` scans the team's entire remaining planned path against all currently blocked edges (not just the new one). If any segment of the path is blocked, A* is immediately re-run from the team's current position to the current target using the updated graph.

The safety check in `move_one_step()` also verifies the next step before executing it: if `get_edge_cost()` returns `inf` for the next cell, the team re-routes rather than teleporting through a blocked edge.

6.4 Team and Civilian Placement

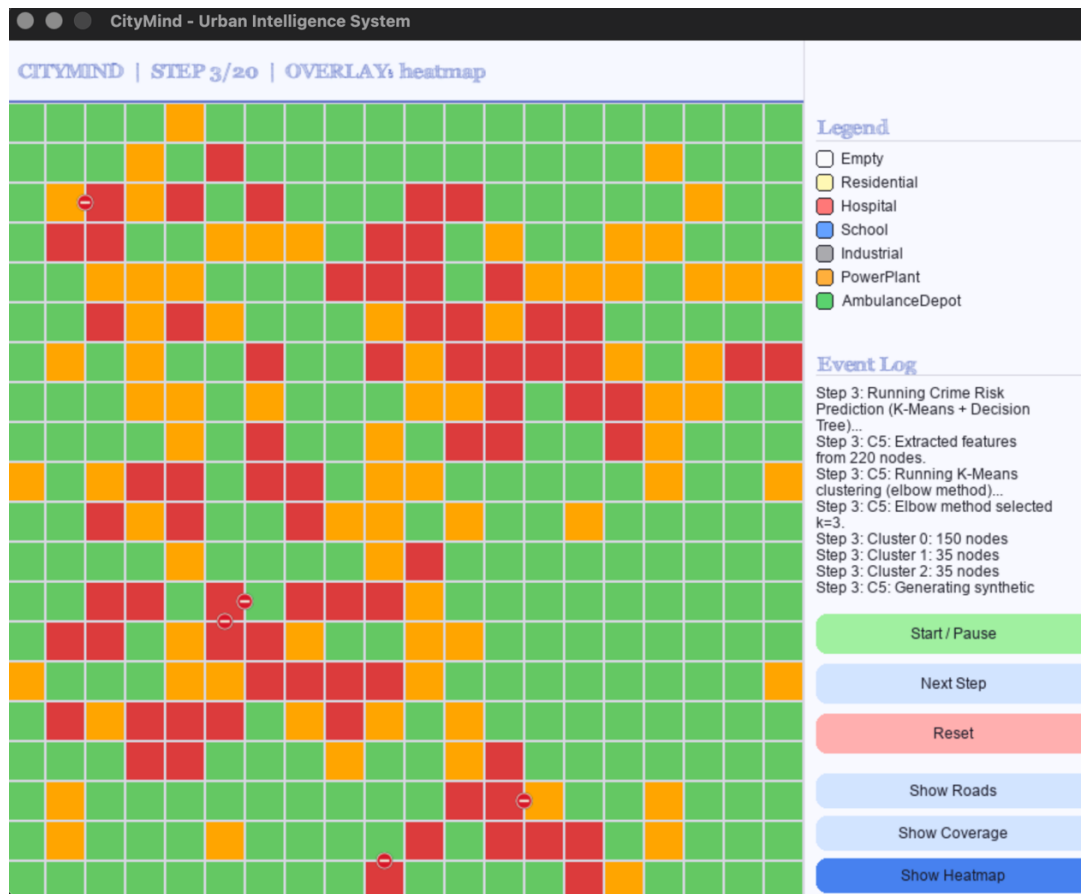
The medical team starts at a randomly chosen Hospital or AmbulanceDepot node. Civilians are placed only on Residential nodes, excluding ambulance positions and the team's starting position. This reflects realistic constraints — trapped civilians would be found in residential areas, not in hospitals or depots.

6.5 Flood System Mechanics

The FloodSystem class operates on edges only and does not modify `node.accessibility`. Blocked edges return `float('inf')` from `get_edge_cost()` and are avoided naturally by both A* and GA without any special handling. Flooded roads are automatically unblocked after 15 seconds. The flood system pauses when the simulation is paused and resumes correctly using timestamp tracking.

7. Challenge 5 — Crime Risk Prediction (K-Means + Decision Tree)

Challenge 5 analyses neighbourhood data across the city, clusters nodes by structural similarity, generates synthetic crime labels, trains a classifier to predict risk levels, and feeds the predictions back into the shared graph as cost multipliers.



7.1 Step 1: K-Means Clustering (Unsupervised)

K-Means groups nodes by feature similarity without requiring any pre-labelled data. Four features are extracted per non-Empty node:

- **population_density**: Scaled to [0,1] across the city distribution.
- **industrial_proximity**: $1 - (\text{min_manhattan_distance_to_industrial} / \text{max_possible_distance})$. Higher score = closer to industrial zones.
- **location_type_score**: A numeric encoding: Industrial = 0.9, PowerPlant = 0.7, Residential = 0.6, School = 0.3, Hospital = 0.2, AmbulanceDepot = 0.2.
- **nearby_blockage_density**: Fraction of built road edges within 2 hops that are currently blocked. This makes the clustering dynamic — flood state influences risk grouping.

K-Means is run with multiple random restarts (5) to avoid poor initialisations, and the best result by inertia is kept. The optimal number of clusters k is selected using the elbow method: K-Means is run for $k = 2$ to 5, and the k where the rate of inertia decrease drops the most (second derivative of the inertia curve) is chosen automatically.

7.2 Step 2: Synthetic Crime Label Generation

Since no real crime data exists for a simulated city, a synthetic dataset is generated using the following documented rule:

Crime Score Formula

```
crime_score = 0.45 × population_density + 0.30 × industrial_proximity + 0.15 ×  
location_type_score + 0.10 × nearby_blockage_density + cluster_noise (small per-cluster random  
offset)
```

This rule is justified: high-density residential areas near industrial zones have higher crime likelihood. Location type acts as a secondary modifier. Cluster noise adds realistic within-cluster variance.

Continuous scores are converted to High/Medium/Low labels by splitting the distribution into equal thirds — the top third becomes High, the middle third Medium, and the bottom third Low. This guarantees a balanced label distribution regardless of the score range.

7.3 Step 3: Decision Tree Classifier (Supervised)

A Decision Tree classifier is trained from scratch using Gini impurity splitting — no external libraries are used. The full feature matrix with synthetic High/Medium/Low labels is split into 75% training and 25% test sets. The tree is built to a maximum depth of 5 with a minimum of 2 samples per split to prevent overfitting. Test accuracy is reported in the event log.

The trained classifier is then applied to all nodes in the city (not just the test set) to produce final risk predictions.

7.4 Graph Integration

Predicted risk labels are mapped to cost multipliers and written to `graph.nodes[pos].risk_index`: High → 1.5x, Medium → 1.2x, Low → 1.0x. Since `get_edge_cost()` multiplies by `risk_index` on every call, both Challenge 3's GA fitness function and Challenge 4's A* search automatically use risk-weighted edge costs from this point forward with no changes to their own code. The `risk_map` is also stored on the graph for the heatmap overlay.

Challenge 5 runs at Step 3 in the simulation — before GA (Step 4) and A* setup (Step 5) — so that both ambulance placement and routing decisions are made on risk-weighted edge costs from the start.

8. Simulation Process

The Simulation class in main.py owns the setup sequence and the per-step loop. The initialisation order is enforced as follows:

Step	Module	What Happens
1	Challenge 1 (CSP)	City layout generated. Node types assigned. Edges initialised with base costs.
2	Challenge 2 (MST + UCS)	Road network built. Flood system activated immediately after.
3	Challenge 5 (ML)	Crime features extracted, K-Means clusters, Decision Tree trained, risk multipliers written to graph.
4	Challenge 3 (GA)	Ambulances placed using risk-weighted BFS fitness function.
5	Challenge 4 (A*)	Civilians placed, team initialised, first A* route planned. Team movement begins.
6+	Rescue Loop	Medical Team moves one cell per interval. Each civilian rescue consumes one simulation step.

Steps 1–5 advance automatically on a 2.5-second timer in auto-run mode. After Step 5, the medical team moves one cell every 1.25 seconds independently of simulation steps, medical team movement is time-based, not step-based. Each civilian rescue consumes one simulation step. The simulation will not advance past Step 5 until all civilians are rescued.

After all civilians are rescued, a wave refresh sequence queues C5 → GA → C4 on the wave_queue. Each queued action consumes one simulation step as the simulation continues to Step 20, re-running crime prediction, ambulance placement, and routing with a fresh set of civilians.

9. User Interface

The visual interface is implemented in Pygame as a single-window application with a 20×20 cell grid on the left and a control panel on the right.

9.1 Grid Display

Each cell is colour-coded by location type. Custom PNG icons replace the default circle/shape markers for the three dynamic entities visible during Challenge 4:

- **Ambulance icon (ambulance.png):** Rendered at each of the 3 ambulance positions after GA placement.
- **Medical team icon (team.png):** Rendered at the team's current grid position during rescue.
- **Civilian icon (civilian.png):** Rendered at each unrescued civilian's position.
- **Blocked road icon (blocked.png):** Rendered at the midpoint of each blocked built road edge.
- **Rescued marker:** A grey X drawn at each position where a civilian has been successfully rescued.
- **A* path:** A yellow line drawn along the team's current planned route.



9.2 Overlay Modes

- **Show Roads:** Draws all MST edges as dark dashed lines, backup UCS edges in purple, and the full backup route in cyan.
- **Show Ambulance Coverage:** Shades each cell based on BFS distance to the nearest ambulance. Darker blue = closer coverage.
- **Show Crime Heatmap:** Shades each cell by predicted risk label after Challenge 5 runs. Red = High, Orange = Medium, Green = Low. Before Challenge 5 runs, cells are shown in neutral grey.

9.3 Control Panel

The right panel contains a Start/Pause button, a Next Step button, a Reset button, and the three overlay toggles. The event log below the buttons displays the most recent simulation events and supports mouse-wheel scrolling to view earlier entries. The top info bar shows the current step, active overlay, and live civilian rescue count (rescued / total) during Challenge 4.

10. Work Division

The work split followed the agreement established in the Phase 1 design document and the group discussion section.

Member	Primary Responsibilities	Notes
Anoosha Ahsan (24I-0831)	Challenge 1 (CSP layout) Challenge 2 (MST + UCS road network) Shared CityGraph class design	Implemented full CSP pipeline including AC-3, backtracking, forward checking, and Min-Conflicts fallback. Implemented metric-closure MST and UCS redundancy phase.
Haris Said (24I-0527)	Challenge 3 (Genetic Algorithm) Challenge 4 (A* routing + flood system) System integration and simulation loop	Implemented full GA pipeline with tournament selection, crossover, mutation, and elitism. Implemented A* with admissible heuristic, dynamic re-routing, and the FloodSystem class.
Both Members	Challenge 5 (K-Means + Decision Tree) UI and visual interface Design document and this report	Challenge 5 was tackled jointly given the shared ML learning curve. UI development was collaborative. All algorithms were built from scratch without external AI/ML libraries.

11. Deviations from the Design Document

The following table documents where the final implementation differs from the Phase 1 design document and justifies each deviation.

Module	Deviation	Justification
Challenge 2	Design doc stated MST on grid-adjacent edges. Implementation uses metric-closure MST where inter-node distances are computed with UCS first.	Grid-edge MST forces roads through every intermediate empty cell. Metric-closure MST finds the cheapest way to connect meaningful locations directly, producing a more realistic and lower-cost road network.
Challenge 3	Design doc described BFS for the fitness function. Implementation uses BFS over built_roads specifically.	Challenge 3 should evaluate coverage along the city's road infrastructure, not through empty land. Using built_roads makes the fitness function consistent with how ambulances would actually travel.
Challenge 4 (routing scope)	Design doc stated A* routes on built roads only. Implementation routes on all grid edges.	Restricting A* to built roads means the team becomes completely stuck when all built roads to a civilian are flooded. Routing over all edges gives the team off-road options in emergencies, which is realistic for emergency vehicles. Blocked edges still return inf and are avoided.

Challenge 5 (execution order)	Design doc numbered Challenge 5 last. Implementation runs it at Step 3, before GA (Step 4) and A* (Step 5).	Risk multipliers from Challenge 5 affect edge costs used by both Challenge 3's fitness function and Challenge 4's A*. Running Challenge 5 first means ambulance placement and routing decisions are made on risk-weighted costs from the start, which is the intended integration.
Challenge 5 (fourth feature)	Design doc described 3 features. Implementation adds a fourth: <code>nearby_blockage_density</code> .	Flood state is a meaningful indicator of area risk. Areas with more blocked roads nearby have reduced emergency access, which correlates with higher effective risk. This makes the clustering genuinely dynamic as the simulation progresses.
UI (PNG icons)	Design doc described plain coloured circles/squares for the team, civilians, and ambulances.	PNG icons (<code>ambulance.png</code> , <code>team.png</code> , <code>civilian.png</code> , <code>blocked.png</code>) were added to make the visual demo more distinct and professional. The shape fallbacks remain in code for environments where the files are missing.